Opportunity Scout AI

CRISP-DM Project Report

Project: Opportunity Scout AI: A Machine Learning-Powered Startup Opportunity Scoring Engine

Methodology: CRISP-DM (Cross-Industry Standard Process for Data Mining)

Team: Vicky Gakuo · Kefa Mwai · Lydiah Khisa · Jeniffer Mbugua · Lilibeth Langat

Capstone Year: 2026

> *"From idea to insight in seconds."* Opportunity Scout AI analyses a startup across market momentum, economic conditions, and funding signals, then returns a calibrated 0–100 opportunity score, full SHAP attribution, a GDP economic backdrop, and actionable improvement recommendations. Everything is explained.

Report Structure

This report follows the six phases of CRISP-DM:

1. **Business Understanding** - What problem are we solving and for whom?
2. **Data Understanding** - What data did we collect, from where, and what did we discover?
3. **Data Preparation** - How did we clean, engineer, and transform raw data into model-ready features?
4. **Modeling** - Which algorithms did we evaluate, what did we choose, and why?
5. **Evaluation** - How well does the model perform, and what does it explain?
6. **Deployment** - How was the model deployed into a live, user-facing application?

Phase 1: Business Understanding

1.1 Background and Motivation

The startup ecosystem is expanding rapidly across both established and emerging markets. In Africa alone, venture capital investment has grown from under $100 million annually in 2010 to billions of dollars by the early 2020s. Yet despite this growth, the process of evaluating whether a startup idea is worth pursuing, or worth funding, remains deeply subjective, inconsistent, and largely inaccessible to those without insider networks or years of investment experience.

Experienced investors rely on pattern recognition built over decades: they can sense when a market is moving, when economic conditions are ripe, and when a founding team is operating in

fertile territory. Entrepreneurs, by contrast, often have no structured way to validate whether their idea is entering a strong or weak market window. The result is a significant information asymmetry and a lot of wasted capital and effort on both sides.

Opportunity Scout AI was built to address this asymmetry. The core insight is that market opportunity is not entirely subjective. It has measurable signals: Google Trends can tell you whether search interest in a domain is rising or falling. The World Bank can tell you whether a country's GDP is expanding or contracting. Reddit community density reveals whether a technology has genuine grassroots interest. News sentiment captured through GDELT signals whether media coverage of a sector is positive or negative. These signals exist; they are public; and they have historically correlated with startup success. The question is whether a machine learning model trained on thousands of historical ventures can learn to combine them into a reliable, explainable opportunity score.

This project set out to answer that question and then to package the answer into a tool that any entrepreneur or investor could use in under a minute, with no technical background required.

1.2 Business Objectives

The project had two primary stakeholder groups, each with distinct but related needs:

Entrepreneurs ask: *"Is this market fertile enough to build in right now? Am I entering at the right time, in the right country, in a sector with rising momentum?"* For this group, the score should emphasise market conditions: trend slope, news sentiment, GDP growth, and community interest. Whether or not external funding has been raised is irrelevant.

Investors ask: *"Has this opportunity earned investor attention? Is external capital already validating the space? Is the market ecosystem mature enough to produce exits?"* For this group, funding validation, ecosystem depth, and capital efficiency matter more. A market that smart money is already entering is a positive signal, not a red flag.

A single, one-size-fits-all score would serve neither group well. A key design objective was therefore to produce **two scoring profiles**, Entrepreneur and Investor, that weight the same eight underlying signals differently based on the perspective of the person asking the question.

Beyond the score itself, the project also needed to explain *why* a score was what it was. A black-box number that says "67/100" is only marginally useful. A system that says "your score is 67, primarily because the trend slope in your market is declining, but your GDP backdrop and news sentiment are strong" gives the user something they can act on.

1.3 Success Criteria

The project would be considered successful if it achieved the following:

- **Predictive quality:** The LightGBM model should achieve an R^2 score above 0.95 on the test set, indicating that the engineered features reliably explain variance in the composite opportunity score.
- **Prediction precision:** Mean Absolute Error (MAE) should be below 0.5 points on a 0–100 scale, ensuring the model is not making wild swings in its estimates.

- **Explainability:** Every score must be accompanied by SHAP (SHapley Additive exPlanations) values showing the top contributing features and their direction of influence.
- **Profile differentiation:** Entrepreneur and Investor profiles must produce meaningfully different scores for the same input, reflecting the genuinely different weightings of the two perspectives.
- **Live signal integration:** The system must attempt to fetch real-time data from four sources (Google Trends, World Bank, Reddit, GDELT) before falling back to historical CSV benchmarks.
- **Deployment:** The full system must be accessible through a web interface that any user can operate without technical knowledge.

1.4 Constraints and Scope

Several constraints shaped the project's design:

- **Training data boundary:** The Crunchbase and Kaggle-derived datasets cover ventures founded between 2005 and 2020. The model was therefore trained on ventures in this window. For ventures founded after 2020, the scoring system evaluates market conditions (which remain timeless) rather than vintage-specific founding factors.
- **Live data reliability:** External APIs - particularly Google Trends, which uses an unofficial interface, can be rate-limited or unavailable. The system was designed with graceful degradation in mind: every signal has a CSV fallback, and a data quality badge tells the user exactly how fresh their data is.

Phase 2: Data Understanding

2.1 Dataset Overview

The foundation of the project is a dataset of **25,018 ventures** sourced from Crunchbase and Kaggle. Each row represents a single startup with its associated market and economic signals at the time of founding. The raw dataset contained 17 columns across four categories: company identifiers and metadata (name, industry, category, country, founded year, status), financial inputs (funding_total_usd, funding_rounds), a pre-computed target variable (opportunity_score and opportunity_category), and seven market/economic signal columns collected via the data pipeline described below.

The opportunity score in the raw dataset was itself constructed using a domain-research-grounded composite formula that weighted market momentum, economic conditions, and funding signals. This formula was the "ground truth" the LightGBM model was trained to learn and generalise. This approach, training a model to predict a well-reasoned composite score rather than a binary success/failure label, was a deliberate design choice. Binary labels for startup success are highly noisy, lagged by years, and highly dependent on factors (like founder personality) that no market signal can capture. A continuous, formula-derived score is far more consistent and learnable.

2.2 The Seven Raw Signals

The pipeline collected seven market and economic signals for each venture, which formed the raw input features before engineering:

1. trend_slope - The slope of Google Trends search interest for the industry keyword over the founding year. A positive slope indicates growing public interest; a negative slope means the topic is cooling. This was computed by fitting a linear regression to the weekly Trends index values for the year in question. Trend slope turned out to be one of the strongest predictors in the model.

2. news_volume - The total count of GDELT-indexed news articles about the relevant industry theme in the target country and year. Higher news volume indicates that the sector is attracting media attention, which tends to correlate with investor and public interest.

3. news_sentiment - The average GDELT AvgTone score for those articles, rescaled from the native GDELT range of (-10, +10) to (0, 6.6) to match the distribution in the training data. Positive sentiment signals a favourable media narrative; negative sentiment can indicate controversy or regulatory concern.

4. reddit_density - A measure of community interest and online discussion density, computed as the number of relevant Reddit submissions per day for the industry keywords in the founding year. This was collected via the PullPush archive API rather than Reddit's live search endpoint. Reddit's own search JSON returns trending posts regardless of keyword — genuinely useless for this purpose. PullPush indexes historical submissions by keyword and timestamp, providing accurate keyword-specific counts for any year.

5. gdp_growth - The year-on-year GDP growth rate for the venture's home country, sourced from the World Bank Open Data API (indicator NY.GDP.MKTP.KD.ZG). This captures the macroeconomic backdrop at the time of founding. A startup entering a market during a GDP contraction faces a structurally harder environment than one entering during a boom.

6. inflation - The annual consumer price inflation rate, also from the World Bank (indicator FP.CPI.TOTL.ZG). High inflation affects both consumer spending power and the cost of capital. Notably, the treatment of inflation differs between the two scoring profiles: for the Investor profile, high inflation is penalised; for the Entrepreneur profile, high inflation is sometimes associated with high-growth emerging markets and was deliberately not inverted.

7. startup_density - A proxy for ecosystem maturity: the number of active startups in the same country and year per some normalised unit. High startup density in the Investor profile is a positive signal (a mature ecosystem with proven exit routes). In the Entrepreneur profile, it is inverted; high density means a saturated market, which is bad for a new entrant.

2.3 Data Collection: The AggregationEngine

The pipeline was built around a class called `AggregationEngine` with two operating modes:

Mode A (Historical): Batch-scores all 25,018 rows from the ventures CSV using stored signal values. Used by the ML team to generate the training dataset.

Mode B (Live): For a new user query, attempts to fetch all seven signals from live APIs in real time. Any signal that cannot be retrieved falls back to the median value from the most similar historical row in ventures.csv (matched by industry, country, and year, with a ±3 year tolerance if an exact match is unavailable).

Each collector was built as a separate class inheriting from `BaseCollector`:

- **GoogleTrendsCollector** - uses `pytrends` to query Google's unofficial Trends API. Rate-limited to 5 calls per 60 seconds because the API aggressively blocks rapid requests. Returns `trend_slope` as the linear regression coefficient over the year's weekly index.
- **WorldBankCollector** - queries the World Bank's free Open Data REST API. No authentication required. Returns `gdp_growth` and `inflation`. Rate-limited to 10 calls per 60 seconds.
- **RedditCollector** - queries the PullPush historical Reddit archive. Rate-limited to 10 calls per 60 seconds. Maps each industry to a keyword list and counts daily submission density for the founding year.
- **GDELTCollector** - queries the GDELT 2.0 dataset via Google BigQuery using a service account key. Returns `news_volume` and `news_sentiment`. This was the most technically complex collector. The free GDELT Doc 2.0 REST API timed out on every live test run; BigQuery proved reliable and fast. Year routing splits queries between two BigQuery tables: `gdelt-bq.gdeltv2.gkg_partitioned` for 2015 and later, and `gdelt-bq.full.events` for 2005–2014.

A **sliding-window rate limiter** (`RateLimiter` class) was implemented for each collector. It tracks the timestamps of recent calls in a deque and blocks (sleeps) if the call budget for the current window is exhausted. This prevents the application from being banned by any of the external APIs during normal use.

A **signal validation** step follows collection. All eight required signals are checked for presence, numeric type, and non-NaN values. Any missing or invalid signal is recorded in an `issues` dict and filled with the median value from the historical dataset.

A **data quality scorer** then computes a quality float from 0.0 to 1.0. Each of the six live-collectable signals contributes to this score: live data earns full credit (1.0 per signal), CSV fallback earns half credit (0.5), and missing data earns none (0.0). The result is labelled High (≥0.75), Medium (≥0.50), or Low (<0.50) and displayed as a badge next to every score in the UI.

An **in-memory cache** (`CACHE` dict) stores results by a composite key of industry + country + year + user_type. Repeated identical queries are served instantly without hitting any external API.

2.4 Exploratory Data Analysis

Before feature engineering, an exploratory analysis of the raw 25,018-row dataset revealed several important patterns:

Funding distribution: Approximately 15% of startups in the dataset had zero external funding ($0). This created a characteristic vertical cluster of points at the zero end of any funding plot. Importantly, these unfunded startups showed a wide range of opportunity scores, confirming that market conditions, not just capital, drive opportunity. This validated the decision to include all seven market signals rather than treating funding as the dominant factor.

Log-funding relationship: A scatter plot of raw funding versus opportunity score showed a weak positive relationship that strengthened considerably after applying a $\log_{10}(x+1)$ transformation. This is the classic right-skewed distribution seen in financial data: the jump from $50K to $500K in funding matters much more than the jump from $50M to $500M. The log transform compresses the scale and makes the relationship approximately linear.

Alt Text

Correlation analysis: A heatmap of all numeric features against opportunity_score confirmed that `trend_slope` had one of the highest correlations, followed by several of the other features. Some economic features had weaker individual correlations, but their power emerged through interaction terms.

Alt Text

Category analysis: A group-by analysis by industry category showed meaningful variation in average scores. Technology, AI, and Finance-related categories tended to score higher on average, while older economy sectors like Government & Politics and Manufacturing scored lower. This validated the decision to include category one-hot encodings as features.

Score distribution: A histogram of opportunity_score showed a broadly normal distribution centred around 50–55, which is consistent with the composite formula's design. There were meaningful tails at both the high end (>70, High Potential) and low end (<40, Lower Potential).

Phase 3: Data Preparation

3.1 Overview

The seven raw signal columns and a handful of categorical fields were transformed into a final feature matrix of **77 model-training features**. The feature engineering was carried out in `feature_engineer.ipynb` and the resulting dataset was saved as `venture_features.csv` (25,018 rows × 86 columns including metadata and target columns; 77 columns used as model input features).

The engineering followed a systematic, hierarchical strategy organised into five categories: scale normalisation, non-linearity capture, interaction terms, categorical encoding, and domain-specific binning.

3.2 Handling Skewness: Log Transforms

Financial and social media data are almost always right-skewed. A startup that raised $500M in funding is an extreme outlier, but in raw form it would exert enormous leverage on any model that treats the feature linearly. Three features received log transforms using `np.log1p(x)` (which safely handles zero values by computing log(x+1)):

- `funding_log` - the natural log of total funding raised
- `news_volume_log` - log of the raw article count from GDELT
- `reddit_density_log` - log of the daily Reddit post density

A separate `log_funding` feature was computed using `np.log10(x+1)` for use in the pipeline scoring formula and SHAP explanations, where a base-10 interpretation is more intuitive to communicate to users.

These transforms and compress the long tails and make the relationships between funding/volume and the target approximately linear, which improves the model's ability to learn them without overfitting to outliers.

3.3 Capturing Non-Linearity: Polynomial Features

Some relationships between features and the opportunity score are not linear. Extreme GDP growth (a boom) and extreme GDP contraction (a recession) may both represent unusual conditions that affect opportunity differently from moderate growth. A squared term captures this U-shape. Three polynomial features were created:

- `trend_slope_sq` - squared trend slope, amplifying the impact of strong positive or strongly negative trends
- `gdp_growth_sq` - squared GDP growth, highlighting extreme economic conditions in either direction
- `funding_sqrt` - square root of total funding, providing a middle scale between the raw value (too skewed) and the log value (too compressed for mid-range funding)

3.4 Interaction Features

Features rarely operate in isolation in the real world. High news volume is a positive signal only when news sentiment is also positive; a market flooded with negative press coverage is not a good environment to launch in. Seven interaction features were created by multiplying related signals together:

- `trend_x_gdp` - trend slope × GDP growth: a rising trend in a growing economy is the ideal combination
- `trend_x_sentiment` - trend slope × news sentiment: captures whether rising search interest is accompanied by positive media coverage
- `reddit_x_sentiment` - Reddit density × news sentiment: combines community enthusiasm with media tone
- `funding_x_rounds` - log funding × funding rounds: measures investor persistence (multiple rounds suggest sustained conviction)
- `trend_x_reddit` - trend slope × log Reddit density: social and search momentum together
- `gdp_x_density` - GDP growth × startup density: economic growth amplified by ecosystem density
- `news_x_sentiment` - log news volume × news sentiment: quality-weighted media signal (volume alone is noisy; volume with positive tone is meaningful)

In addition to these multiplicative interactions, two composite derived features were created:

- `econ_health` - GDP growth minus inflation, representing real economic growth adjusted for price pressure

- `market_validation` - a weighted composite of trend slope (40%), normalised Reddit density (30%), and normalised news sentiment (30%), designed as a single-number proxy for whether the market is validating the opportunity from multiple angles simultaneously

3.5 Ratio Features

Ratios capture efficiency and relative relationships that raw values miss:

- `funding_per_round` - total funding divided by (number of rounds + 1): high values indicate institutional investors are writing large cheques per round, signalling conviction
- `real_gdp_growth` - GDP growth divided by (1 + inflation/100): the true inflation-adjusted growth rate
- `investment_climate` - GDP growth divided by (inflation + 1): a compact macro efficiency ratio
- `ecosystem_strength` - startup density × GDP growth / 100: a combined proxy for how hospitable the overall environment is to startups

3.6 Z-Score Normalisation

Z-scores were computed for the three most variance-heavy continuous signals to ensure all features share a common mean (0) and standard deviation (1), preventing the model from being biased toward features with larger raw scales:

- `trend_zscore` - normalised trend slope
- `gdp_zscore` - normalised GDP growth
- `reddit_zscore` - normalised Reddit density

The mean and standard deviation parameters were saved to `zscore_params.csv` at training time. The `VentureFeatureEngineer` class loads these parameters at runtime and applies the same normalisation to live inputs, ensuring that new predictions are scaled consistently with the training data, a critical detail for avoiding data leakage.

3.7 Categorical Encoding

Machine learning models cannot process text directly. Three categorical columns were encoded:

Industry/Category (26 sectors): One-hot encoded using `pd.get_dummies`, creating 26 binary columns prefixed with `cat_`. The 26 sectors ranged from `cat_AI & Machine Learning` and `cat_Finance & Fintech` through to `cat_Government & Politics` and `cat_Sports & Recreation`. This allowed the model to learn sector-specific opportunity patterns.

Company Status (3 states): One-hot encoded into `status_acquired`, `status_closed`, and `status_operating`. In the live prediction context, new ventures are always assigned `status_operating = 1`.

Country (frequency grouping): Encoding 103 countries with one-hot encoding would have created the curse of dimensionality, sparse binary columns for rarely-seen countries that the model could never learn from meaningfully. Instead, the top 10 countries by venture count were

identified (United States, United Kingdom, Canada, Germany, France, India, China, Israel, Spain, Singapore) and encoded individually. All remaining 93+ countries were grouped into a single `country_Other` column. This reduced the country feature space to 11 columns while preserving all meaningful geographic signals.

3.8 Funding Tiers

A domain-specific binning was applied to total funding to create a qualitative "stage of company" feature:

- `tier_unfunded` - 0 raised
- `tier_seed` - 1 to 2,000,000
- `tier_series_a` - 2,000,001 to 10,000,000
- `tier_series_b_plus` - above 10,000,000

These tiers capture qualitatively different risk and validation profiles. A bootstrapped startup has no external validation; a seed-stage company has some; a Series B company has multiple rounds of institutional validation. The model could theoretically learn the same relationship from raw or log funding, but discrete tiers make this easier and more interpretable.

3.9 Final Checks

After all transformations, the dataset was verified to have zero null values across all 77 model features. The z-score parameters were exported to `zscore_params.csv` for use at prediction time. The complete engineered dataset was saved as `venture_features.csv`.

Phase 4: Modeling

4.1 Model Selection: The Four Candidates

Four regression algorithms were evaluated in the first round of training, all configured with reasonable default hyperparameters and evaluated on an 80/20 train/test split (20,014 training samples, 5,004 test samples):

Model	MAE	R^2 Score	Training Time
Gradient Boosting	0.1251	0.9973	40.16 s
XGBoost	0.1283	0.9973	2.34 s
LightGBM	**0.1508**	**0.9981**	**0.88 s**
Random Forest	0.1872	0.9948	12.29 s

The results immediately showed that all four algorithms performed exceptionally well on this dataset - R^2 scores above 0.99 across the board indicate that the engineered features are highly predictive of the composite opportunity score. This is expected: the score was itself derived from these signals, and the model is learning a well-structured formula. The question was therefore

not "which model is accurate enough" but "which model best balances accuracy, speed, and deployability."

LightGBM was selected for the following reasons:

Speed: LightGBM trained in 0.88 seconds - roughly 46× faster than Gradient Boosting and 14× faster than Random Forest. In a live web application where a user is waiting for a score, inference time matters. LightGBM's leaf-wise tree growth and histogram-based binning make it significantly faster than node-wise boosting methods.

R^2 Score: LightGBM achieved the highest R^2 score (0.9981) of any model, meaning it explains 99.81% of the variance in the opportunity score. This slightly higher R^2 compared to XGBoost and Gradient Boosting reflects LightGBM's ability to capture fine-grained patterns through its num_leaves parameter controlling tree complexity rather than max_depth.

Production suitability: LightGBM produces compact model files, has native support for SHAP explainability through `shap.TreeExplainer`, and integrates cleanly with the FastAPI + joblib deployment stack. It also handled whitespace in feature names (from the category one-hot columns) gracefully with a simple column-cleaning step.

MAE note: LightGBM's MAE (0.1508) is slightly higher than Gradient Boosting (0.1251) and XGBoost (0.1283). This is a minor trade-off; on a 0–100 scale, a difference of 0.025 MAE points is negligible for practical use. The speed and R^2 advantages more than justify accepting this minor precision difference.

4.2 Hyperparameter Tuning

After the initial comparison, the two strongest models (LightGBM and XGBoost) were subjected to hyperparameter search using `HalvingRandomSearchCV` with 3-fold cross-validation. The halving search progressively allocates more resources to promising parameter combinations and fewer to weak ones, making it significantly faster than standard `RandomizedSearchCV` or `GridSearchCV`.

The search space for LightGBM covered:

- `n_estimators`: 100–300 (number of trees)
- `learning_rate`: 0.01–0.11 (step size per tree)
- `num_leaves`: 20–60 (primary complexity control the key LightGBM hyperparameter)
- `feature_fraction`: 0.70–1.00 (column subsampling per tree)
- `bagging_fraction`: 0.70–1.00 (row subsampling per iteration)

The best LightGBM configuration found by the search:

- `bagging_fraction`: 0.7026
- `feature_fraction`: 0.7060
- `learning_rate`: 0.0687
- `n_estimators`: 289
- `num_leaves`: 38
- **Cross-validated MAE: 0.1527**

4.3 Final Model Training

After tuning, the final model was trained on **100% of the data** (train + test combined, 25,018 rows) using the best hyperparameters.

The final trained model was serialised using `joblib.dump` into `ventures_lightgbm.joblib`, which also stores the feature names list and z-score parameters for use at prediction time.

Final model metrics on the held-out test set (evaluated before retraining on full data):

- **Test MAE: 0.0974**
- **Test R^2: 0.9994**

The model explains 99.94% of the variance in the opportunity score with an average prediction error of under 0.1 points on a 0–100 scale.

4.4 The Opportunity Score Formula and Dual Profiles

A core modeling decision was how to differentiate scores by user perspective. The LightGBM model itself is profile-agnostic; it was trained once on the full dataset and produces the same raw prediction regardless of whether the user is an entrepreneur or an investor. Profile differentiation is introduced through a **score blending** mechanism in the production predictor.

The pipeline's `AggregationEngine.run_live()` computes a separate **pipeline-weighted score** using explicit, handcrafted weight profiles for each user type. These weights were grounded in the exploratory data analysis and domain research:

Entrepreneur weights (asking: "Is this market fertile to build in?"):

- trend_slope: 35% — strongest predictor for bootstrapped startups
- news_sentiment: 25% — market mood without funding as backstop
- gdp_growth: 15% — macro conditions matter when entering a market
- reddit_density: 12% — community interest
- news_volume: 8%
- inflation: 2%
- startup_density: 3% (inverted — high density = saturated = bad for new entrants)
- log_funding: 0% — meaningless; entrepreneur has no funding yet

Investor weights (asking: "Has this company earned its score?"):

- log_funding: 22% — other smart money already validated the space
- trend_slope: 20%
- news_sentiment: 18%
- news_volume: 10%
- reddit_density: 10%
- gdp_growth: 10%
- startup_density: 8% (not inverted — a mature ecosystem with exits is good for investors)
- inflation: 2%

The final score returned to the user is a **weighted blend** of the model's raw prediction and the pipeline-weighted score: Entrepreneur receives 60% model + 40% pipeline; Investor receives 78% model + 22% pipeline. This blend preserves the statistical rigour of the LightGBM model while introducing meaningful profile-based differentiation.

4.5 Score Calibration

The raw LightGBM output is a continuous float on approximately the 0–100 scale of the training target. A `calibrate_score()` function clips this to [0, 100], rounds it to two decimal places, and assigns a tier and confidence level:

- **High Potential** (≥70): 90% confidence - clear signals of strong opportunity
- **Medium Potential** (40–69): 85% confidence - reasonable opportunity with some areas for improvement
- **Lower Potential** (<40): 75% confidence - significant headwinds; score is less certain

A ±4 point confidence range is also returned, reflecting the model's expected margin of error and preventing users from over-indexing on the specific decimal value.

Phase 5: Evaluation

5.1 Quantitative Performance

The LightGBM model's test-set performance figures represent excellent predictive quality by any standard:

Test MAE: 0.0974 - On a 0–100 scale, the model's predictions deviate from the ground-truth composite score by less than one-tenth of a point on average. For context, the score range spans 100 points, and a user would need to see a difference of at least 3–5 points to notice any practical change. An MAE of 0.0974 means the model is effectively memorising the score formula to very high precision.

Test R^2: 0.9994 - The model explains 99.94% of the variance in opportunity scores across 5,004 unseen test cases. This extremely high R^2 reflects the fact that the training target (opportunity score) was itself derived from the same signals used as features, just via a different mathematical form. The model is learning a representation of that formula and generalising it across industry, country, and year combinations it has seen.

Prediction speed: - The trained LightGBM model produces a prediction and SHAP values for a single input in under 1 second, well within the web application's performance requirements.

5.2 SHAP Explainability

SHAP (SHapley Additive exPlanations) was applied using `shap.TreeExplainer`, which computes exact Shapley values for tree-based models efficiently. SHAP values assign to each feature the exact number of score points it contributed (positively or negatively) to the model's prediction for that specific input.

The top 5 features by mean absolute SHAP value across the test set were:

1. **funding_total_usd** - mean |SHAP|: 1.6465
2. **market_validation** - mean |SHAP|: 1.6438
3. **trend_slope** - mean |SHAP|: 0.9581
4. **trend_x_sentiment** - mean |SHAP|: 0.6331
5. **log_funding** - mean |SHAP|: 0.5926

Alt Text

These findings confirm the domain logic embedded in the feature design: market momentum (trend_slope and market_validation) and funding validation (funding_total_usd and log_funding) are the two dominant signal families. The interaction term trend_x_sentiment captures a sophisticated relationship: a rising market with positive press coverage scores significantly better than a rising market with negative coverage.

SHAP in the production app: For every live prediction, the SHAP explainer computes values for all 77 features and returns the top 10 by absolute impact, suppressing any time-based features (which may appear as residual entries in the model's feature list). Each factor is presented to the user with its feature name, SHAP value, direction (positive = boosting, negative = dragging), and an explanation of what the feature measures and why it is moving in that direction for their specific input.

5.3 Success Pattern Analysis

A success pattern detection analysis was run to identify which features most differentiated accurate predictions from inaccurate ones. The analysis split the 5,004 test cases into the 25% with the smallest prediction errors (successful) and 75% with larger errors (unsuccessful), then compared mean absolute SHAP values between the groups.

Features that contributed more to successful predictions (stable, learnable patterns):

- funding_total_usd (+0.3606 higher importance in success cases)
- log_funding (+0.1159)
- funding_per_round (+0.0541)

Features that contributed more to unsuccessful predictions (noisier or harder cases):

- market_validation (-0.0882 cases where the composite market signal conflicted with other factors)
- trend_slope (-0.1070)

Alt Text

This analysis suggests that the model is most confident when funding signals are clear and consistent. Cases involving ambiguous or conflicting market momentum signals produce more prediction error, which is consistent with the real-world observation that market timing is genuinely hard to predict.

5.4 The GDP Context Engine

A supplementary evaluation component was the GDP Context Engine, which adds a qualitative economic narrative to every score. This was motivated by the observation that a score of 55 in Kenya 2019 means something very different from a score of 55 in the United States 2019; the macroeconomic contexts are entirely different, and a user who doesn't already know this context is missing important information.

The engine was implemented as a hybrid lookup system:

- A **static table of 168 curated country-year GDP events** spanning 2005–2020, hand-written to cover every significant GDP movement in the training data (financial crises, commodity booms, debt crises, post-crisis rebounds, etc.)
- An **Anthropic Claude API fallback** that generates a one-sentence explanation on the fly for any country-year combination not in the static table
- A **graceful degradation path** that produces a plain factual sentence if both the table and the API are unavailable

The GDP context is displayed as a green-highlighted panel in the score results, framing the user's score within the economic conditions their venture would have faced.

5.5 Similarity Matching

A cosine similarity matching component was also implemented during evaluation. For each test-set prediction, the system identifies the top-5 most similar ventures in the training set (using StandardScaler-normalised features and cosine similarity), then compares their predicted scores and SHAP patterns. This can be used to surface benchmark ventures, real historical companies that operated in similar conditions, to contextualise a new prediction.

In the production application, this functionality is surfaced through the Benchmark section of each score result, which shows the number of higher-scoring historical ventures in the same industry category as a reference point.

Phase 6: Deployment

6.1 Architecture Overview

The production system consists of four layers, each with a clearly defined responsibility:

```
Browser (HTML/CSS/JS)
        ↕ HTTP (REST)
FastAPI Backend (main.py)
        ↕ Python function calls
Predictor (predictor.py)
        ↕
Pipeline (pipeline.py) ←→ Live APIs + ventures.csv
        ↕
```

```
LightGBM Model (ventures_lightgbm.joblib)
        ↕
SHAP Explainer
```

The system runs locally using `uvicorn` as the ASGI server. The frontend is a vanilla HTML/CSS/JavaScript single-page application served as static files. The app is hosted live on Contabo for users to easily access it. Check it out on this link

6.2 Backend: FastAPI

The backend (`main.py`) is a FastAPI application with the following endpoints:

- `POST /evaluate` - the primary endpoint. Accepts a JSON body with venture name, industry, country, founded year, total funding, funding rounds, and user_type. Calls `predictor.predict()` and returns the full result dictionary.
- `GET /api/v1/industries` - returns the list of 25 supported industry categories for populating the frontend dropdowns.
- `GET /api/v1/countries` - returns the list of supported countries.
- `GET /health` - returns the API status, displayed as a live dot in the top-right corner of the UI.

The `OpportunityPredictor` class is instantiated once at application startup and shared across all requests. This means the 25,018-row reference DataFrame and the SHAP TreeExplainer are loaded into memory once and reused. LightGBM SHAP computation can be expensive to initialise, and loading it per-request would be unacceptably slow.

Input validation is handled by Pydantic models. Boundary checks are also enforced on the frontend before the API call is made.

6.3 Predictor Layer

`predictor.py` is the bridge between the FastAPI layer and the pipeline/model layers. Its `predict()` method:

1. Calls `pipeline.run_live()` to collect live signals and compute the profile-weighted pipeline score.
2. Seeds a feature row by finding the nearest reference row in `venture_features.csv` for the given industry and year, then overwrites it with the user's specific inputs (funding, country, and founded year) and the live signals.
3. Passes the row through `VentureFeatureEngineer.transform()` to compute all 77 engineered features.
4. Runs `model.predict()` to get the raw LightGBM prediction.
5. Blends the model prediction with the pipeline score according to the user_type profile.
6. Calibrates the blended score to a 0–100 range and assigns a tier.
7. Computes SHAP values for the input row, filters out suppressed time features, and builds the top-10 factor list with explanations.
8. Generates recommendations for any negative factors with |SHAP| > 0.3.
9. Computes benchmark statistics against the reference dataset.

10. Returns the complete result dictionary to FastAPI.

6.4 Frontend Application

The frontend is a single-page application built in vanilla HTML, CSS, and JavaScript with no frameworks or build toolchain. It was deliberately kept simple to maximise maintainability and avoid dependency management complexity. The application has two views:

Landing Page: A marketing-style overview of the product, including the methodology note, signal sources, and a live sample score card showing a Kenya Fintech Startup example with GDP context. Navigation links scroll to product features, process explanation, and the About section. The footer credits the full team: Vicky, Kefa, Lydiah, Jeniffer, and Lilibeth.

App Interface: Three tabbed panels:

Score Tab

The primary interface. The user selects a profile (Investor or Entrepreneur), enters a venture name, chooses an industry and country from dropdowns, and provides a founded year, total funding, and number of funding rounds. The profile toggle is the first element in the form, immediately communicating to the user that the scoring perspective matters. A hint below the toggle explains exactly how each profile weights the signals differently.

Upon submission, the result panel displays:

- A circular SVG score ring with the numeric score and tier colour (green ≥70, amber 40–69, red <40)
- Confidence percentage, score range, base SHAP value, percentile ranking, and category average
- A profile badge (Investor or Entrepreneur) identifying which perspective was used
- A GDP Context panel (green card) with a one-sentence economic backdrop for the country and year
- A data quality badge indicating how many signals came from live APIs vs historical fallback
- Top-10 SHAP factor cards with icons, colour-coded left borders (green = boosting, red = dragging), gradient fill bars showing relative impact, and plain-English explanations
- Up to 4 actionable recommendations targeting the lowest-scoring factors, each with a specific problem statement, recommended action, and estimated potential score gain

What-If Tab

Allows users to simulate the impact of changing funding or founding year on their score. The user specifies a base scenario (industry, country, year, funding) and a profile, then adjusts two sliders: one for funding (0 to 50M) and one for year (2000–2030). Clicking "Run Scenario" makes two simultaneous API calls, one for the base and one for the modified scenario, and displays both scores side by side with a delta indicator showing the exact point change. Full SHAP explanations for both scenarios are shown below.

Compare Tab

Allows side-by-side evaluation of up to 5 opportunities simultaneously. A single profile toggle at the top applies the same perspective to all comparisons. Dropdowns for each card are populated asynchronously after the industry/country lists are fetched from the API. A "Winner" crown badge identifies the highest-scoring opportunity.

6.5 Data Quality Transparency

A deliberate design decision was to surface data quality prominently rather than hide it. Every score result shows:

- A coloured quality dot: green (High — most signals live), amber (Medium — mixed live and fallback), or red (Low — mostly CSV fallback)
- A "live signals" badge showing how many of the six collectable signals were retrieved in real time
- In the What-If and Compare results, the same badge appears per scenario

This transparency is important for honest use of the tool. A score based entirely on 2015 historical CSV data for a country that the live APIs couldn't reach is less trustworthy than one based on five live signals retrieved seconds ago. Users deserve to know the difference.

6.6 Unit Tests

The pipeline was accompanied by a `TestPipeline` unit test suite covering 12 test cases:

- Signal validation with valid, missing, NaN, and type-error inputs
- Scorer output range (0–100 guaranteed)
- VenturesCollector with an impossible industry/country/year
- `run_live()` mode field verification
- `run_historical()` DataFrame schema and row count
- Data quality scoring for all-live and all-fallback signal sets
- Rate limiter non-blocking behaviour for sub-limit bursts
- Output dictionary containing all required quality and context fields

All 12 tests passed, confirming the pipeline's core behaviour is correct and resilient.

Limitations and Future Work

Known Limitations

Training data boundary: The model was trained on ventures founded between 2005 and 2020. Time features were deliberately removed to prevent out-of-distribution penalties for current-year ventures, but the model's learned patterns still reflect the market dynamics of that era. A sector that emerged after 2020 (e.g., large language model infrastructure) has no training examples and will be scored based on proxy features from adjacent categories.

Ground truth construction: The opportunity score used as the training target is a researcher-constructed composite formula, not an objective measure of "real" startup success. Actual startup success depends on factors the model cannot observe: founder quality, team dynamics, execution, timing, luck, and network access. The score should be understood as a market opportunity index, not a startup success predictor.

Google Trends rate limiting: The unofficial `pytrends` library is frequently rate-limited by Google, particularly when querying multiple times in quick succession. In a session with many score requests, trend_slope may fall back to historical CSV values for some queries.

Geographic coverage: The WorldBankCollector covers countries with ISO codes in its map. Many smaller countries, particularly in Sub-Saharan Africa, Southeast Asia, and Central America, fall back on CSV data for GDP and inflation. Expanding the ISO code map would improve live data coverage for these markets.

Future Work

Model retraining pipeline: A scheduled retraining job that updates the model quarterly with new ventures from the Crunchbase dataset would keep the learned patterns current and eventually extend the training boundary beyond 2020.

Additional live signals: Patent filing activity, LinkedIn talent density (through the LinkedIn API), and app store ranking trends are examples of signals that could augment the current eight. Each would require a new Collector class, a fallback CSV value, and validation in the feature engineering step.

Expanded GDP context coverage: The static GDP table covers 168 country-year events. The Anthropic API fallback handles the rest, but extending the static table to cover more of the dataset would reduce API dependency and improve response time.

Team size context: The scoring system currently ignores team composition. A future version could accept team size, domain expertise years, and previous exit history as optional inputs, using them to adjust the score in a separate "team quality" dimension.

Conclusion

Opportunity Scout AI demonstrates what is possible when rigorous data science methodology is paired with a clear, user-focused product vision.

The CRISP-DM process guided every decision: a clearly defined business problem (information asymmetry between investors and entrepreneurs) led to a specific data collection strategy (four live API collectors with graceful fallback), which led to a principled feature engineering process (77 features across five engineering categories), which led to a well-evaluated model (LightGBM, R^2 0.9994, MAE 0.0974), which led to a deployed application that any user can operate in under a minute.

What makes Opportunity Scout AI more than a model evaluation exercise is the attention paid to deployment. The GDP Context Engine adds economic literacy to a raw number. The dual scoring profiles acknowledge that entrepreneurs and investors are asking genuinely different questions.

The data quality badge tells users exactly how much to trust their score. The SHAP explanations make every prediction auditable and actionable. And the What-If simulator turns a passive score into an active planning tool.

The system was built by a team of five for a 2026 capstone project, trained on 25,000+ historical ventures, and deployed as a fully functional web application. It is, as the landing page says, built with ML rigour, designed for clarity.

Team: Vicky · Kefa · Lydiah · Jeniffer · Lilibeth | **Capstone 2026** | **Powered by LightGBM + SHAP**